



中山大學
SUN YAT-SEN UNIVERSITY

文件操作

學大山中立國
中山大学计算机学院



讲课人: 陈武辉

目录

CONTENTS

01

文件概述

02

文件基础操作

03

文本文件处理

04

二进制文件处理

05

文件随机读写



文件概述

文件(File): (狭义地) 是指存储在计算机外部介质 (如磁盘等) 上的文字序列或二进制字节的序列。

磁盘文件指保存在磁盘上的文件, 它有唯一的**全称** 例如: `"/sources/hello.c"`, 其中`"/sources/"`是文件**路径***path*, `"hello.c"` 是**文件名***name*; 文件**长度**用字节数表示。

文件的用途:

1. **持久保存。**例如编写的源代码, 编译产生的中间件结果和执行文件能长期保存;
2. **数据共享。**多个程序运行时可以使用相同的数据, 或某程序产生数据供其他程序使用等;
3. **数据与程序分离。**数据的改变不会导致程序重新编译、运行。

文件概述

以数据为中心视角，文件可分两大类：

文本文件(text)： 由ASCII码及其扩展集构成字符构成的文字序列。

- 可以使用任何文本编辑软件打开，如记事本，Sublime，vs code 等等；
- 编码属性，如 ascii，ansi（含 GBK，BIG5等），utf-8等；
- 若其中包含 EOT(4), STX(3), -1 等控制符在部分系统中会导致文件传输中止。

二进制文件： 以字符数组格式存放的数据。

- 每类应用有独特的数据，例如 .zip只能由压缩解压程序处理，.obj只能由编译程序使用；
- 通常是将内存中数据格式原样输出到文件中，例如整数会以4个字节补码格式存储；
- 保存struct 结构体时需要考虑 packed 或 unpack 情况，在空间和性能之间平衡。



文件概述

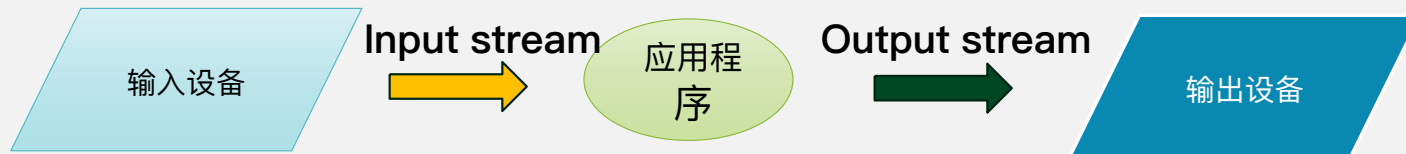
以应用为中心的视角，文件分类：

输入流(stream)：字符或数据通过类似管道的机制流入应用的字节流。流出应用的称为**输出流**。

能产生输入流的设备称为输入设备，如键盘、鼠标、传感器等等。

同样，显示器、打印机、控制器等称为输出设备。

如磁盘等既能产生输入，又接受输出，则称为输入输出设备。



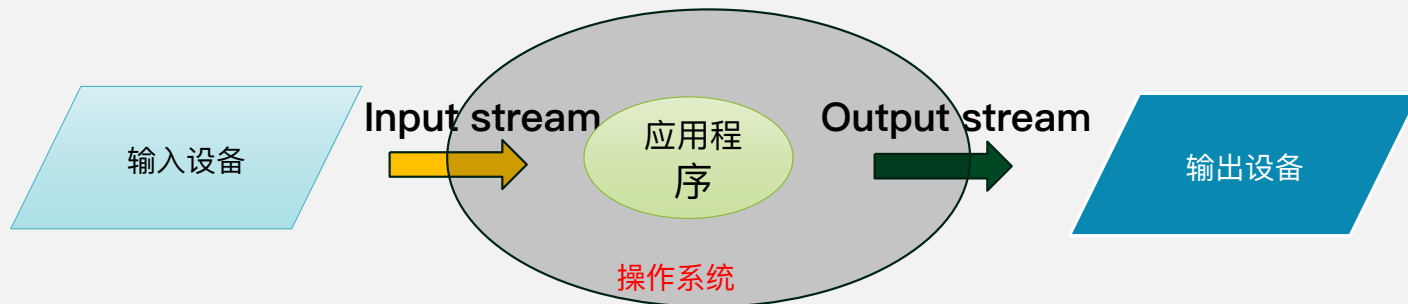


文件概述

以操作为中心的视角，文件操作是操作系统提供的关于流的**通用操作**：

应用程序不能直接操作输入或输出设备。必须通过操作系统完成文件操作。

1. **打开文件**：通过一个文件标识符（文件名）建立设备与应用之间的流机制。
2. **操作文件**：**读文件** 从流管道中**顺序读入**文字或字节；**写文件** 将字符串或字符数组**顺序输出**到流管道中。
3. **关闭文件**：通知操作系统关闭流机制，释放该文件占用的各种资源。





文件基础操作 (1)

头文件<stdio.h>给出了C语言与操作系统交互的函数与数据结构：

1. **结构体 FILE**：(不是 struct FILE)描述了控制与管理 IO 流需要的信息。应用程序不建议直接操作结构体，应使用 FILE * 指针通过相关函数操作文件。

2. **预定义标准流**：

- **stdin** 是 FILE * 变量，标准输入流。默认是控制台键盘输入
- **stdout** 是 FILE * 变量，标准输出流。默认是控制台输出显示设备
- **stderr** 是 FILE * 变量，标准错误输出流。默认是控制台输出显示设备
-

注意：预定义标准流是操作系统为应用程序准备好的流，应用程序一般不需要关闭或打开，可直接读入或输出。例如：scanf 使用 stdin 读入数据，printf 使用 stdout 输出文本。



文件基础操作 (2)

头文件<stdio.h>给出了C语言与操作系统交互的函数与数据结构：

3. 函数（红色需要掌握，黑色了解）：

- 文件访问：fopen, fclose, freopen, fflush, ...
- 直接输入/输出：fread, fwrite, ...
- 无格式文本输入/输出：fgets, fputs, fgetc/getc, fputc/putc, ungetc
；标准输入/输出：getchar, putchar, gets, puts；
- 格式化文本输入/输出：fscanf, fprintf；标准输入/输出：scanf, printf；字符串输入/输出：sscanf, sprintf；...
- 文件位置：fseek, ftell, fgetpos, fsetpos, rewind；



文件基础操作 (3)

头文件<stdio.h>给出了C语言与操作系统交互的函数与数据结构：

3. 函数（红色需要掌握，黑色了解）：

- 错误处理：clearerr, **fEOF**, ferror, perror;
- 文件操作：remove, rename, tmpfile, tmpnam;

4. 宏常量

- **EOF**



文件基础操作 – 步骤

C语言中文件操作一般有四个步骤：

1. 定义处理文件的变量：

- 例如：FILE *dest, *src;

2. 打开文件：

- 例如：dest = fopen("1.txt", "w");
- 注意：必须检查文件指针是否为空

3. 读写文件，或调整读写位置。注意文本与二进制读写的区别。

4. 关闭文件：

- 例如：fclose(dest);



文件基础操作 – 案例研究

复制磁盘上的文件：

任何磁盘文件可看作字符数组

以写方式与dest文件建立流d

如果有错误，文件指针为 NULL

以读方式与scr文件建立流s

当文件没读完

从流s顺序读最多4K字节到buf

将buf中cnt个字节顺序写入流d

关闭流s和d

```
/*file copy*/
int FileCopy(char *dest, char *scr) {
    char buf[1024*4];

    FILE *d = fopen(dest,"wb");
    if (!d) return -1;
    FILE *s = fopen(scr,"rb");
    if (!s) return -1;

    while (!feof(s)) {
        size_t cnt = fread(buf,1,sizeof(buf),s);
        fwrite(buf,1,cnt,d);
    }

    fclose(s);
    fclose(d);
    return 0;
}
```

文件基础操作 – 打开与关闭文件

打开文件：

FILE *fopen(const char *filename, const char *mode);

- 打开 filename 所指示的文件，并返回指向关联到该文件的文件流的指针。mode 用于确定文件访问模式。
- 若错误返回空指针，可通过外部变量 errno 获得错误编号，strerror() 获得错误文本。

关闭文件：

- 关闭并释放文件流。写入任何未写入的缓冲数据到 OS 。舍弃任何未读取的缓冲数据。
int fclose(FILE *stream);

文件基础操作 – 打开方式

打开文件方式： mode 字符串

文件访问模式字符串	含义	解释	若文件已存在的动作	若文件不存在的动作
"r"	读	打开文件以读取	从头读	打开失败
"w"	写	创建文件以写入	销毁内容	创建新文件
"a"	后附	后附到文件	写到结尾	创建新文件
"r+"	读扩展	打开文件以读/写	从头读	错误
"w+"	写扩展	创建文件以读/写	销毁内容	创建新文件
"a+"	后附扩展	打开文件以读/写	写到结尾	创建新文件

文件访问模式标签 "b" 可以可选地指定以二进制模式打开文件。此标签仅在 Windows 系统上生效。在附加文件访问模式下，数据被写入到文件尾，而不考虑文件位置指示器的当前位置。

当文件以更新模式打开时（ '+' 作为上述模式参数值列表的第二或第三个字符），访问流可以进行输入和输出。然



文件基础操作 – 直接读或写

读数据:

size_t fread(void *buffer, size_t size, size_t count, FILE *stream);

- 从给定输入流 stream 读取至多 count 个 size 字节的对象到数组 buffer 中。
- 成功读取的对象数，若出现错误或文件尾条件，则可能小于 count 。
- fread 不区别文件尾和错误，而调用者必须用 feof 和 ferror 鉴别出现者为何。

写数据

size_t fwrite(const void *buffer, size_t size, size_t count, FILE *stream);

- 写 count 个来自给定数组 buffer 的对象到输出流 stream。
- 成功写入的对象数，若错误发生则可能小于 count 。



文件基础操作 – 文件结束？

检查文件结束：

```
int feof( FILE *stream );
```

检查是否已抵达给定文件流的结尾。

知识：对于磁盘文件

，可以看作二进制的动态字符数组。当文件以读方式打开时，有一个读写装置放在待读字节的位置（开始位置），当收到读指令，则将当前字节读了流，然后下移一格顺序读，当读完后指向添加位置时，则就是所谓文件尾部，这时 feof 返回 非零值。



读写头



文本文件操作

文本文件具有通用、易于阅读、编辑方便等优势，在 Unix

操作系统中，用户名、网络配置等都采用文本文件保存。文本流处理在C语言标准库中比较丰富。

1. 文本由行构成，因此文本流默认是行缓冲，即遇到换行符文本处理函数才会启动：

- 例如：getchar 虽是逐一读入字符，但字符流保存在行缓冲中直到换行或流结束；

2. 在绝大多数情形下，文本流仅需要顺序读写，即打开模式是“r”，“w”，“a”之一：

- 例如：dest = fopen("1.txt", "a"); 在1.txt文件后添加内容；

3. 部分文本流可能不存在结束，即 feof() 函数永远为假：

- 例如：键盘输入流；

4. 文本文件操作分为有格式文本操作函数和无格式文本操作函数。



文本文件操作 – 有格式文本

格式化文本由一行或多行描述一个数据条目，如学生信息。每个数据条目中包含若干数据域，数据域之间用空白字符分隔。例如：

```
/*text formatted*/
int main() {
    FILE *pf = fopen("student.txt","r");
    if (!pf) {
        perror("Open File:");
        exit(9);
    }
    // 练习1：这里有三个版本的 stReader，它们都对吗？
    // 练习2：将参数改为 stdin 会发生什么？
    stReader(pf);
    fclose(pf);
}
```

```
void stReader(FILE *pf) {
    printf("学生信息: \n");
    int id;
    char name[32],college[32];
    double score;
    while (!feof(pf)) {
        fscanf(pf,"%d%s%s%lf",
               &id,name,college,&score);
        printf("学号%d 姓名%s 学院%s 成绩\n",
               id,name,college,score);
    }
}
```

文本文件操作 – 格式化读或写

读数据：

int fscanf(FILE *stream, const char *format, ...);

- 熟悉的 **scanf** 等价于 **fscanf(stdin, format, ...);**
- **返回值** 成功接收参数的数量（可以为零），或者若输入首个参数赋值前发生失败则为 **EOF** 。
- 如何从键盘这类无尽文本流读数据，必须思考合适的终止方法。

写数据

int fprintf(FILE *stream, const char *format, ...);

- 熟悉的 **printf** 等价于 **fprintf(stdout, format, ...);**。
- **返回值** 输出的字符数，或若出现输出错误或编码错误则为 **负值**



文本文件操作 – 无格式文本

无格式化文本是看作由**字符串或字符**构成的文本。这类文本通常先读入内存，再进行处理。例如：

```
/*text normal*/
int main(void) {
    FILE* tmpf = tmpfile();
    fputs("Alan Turing\n", tmpf);
    fputs("John von Neumann\n", tmpf);
    fputs("Alonzo Church\n", tmpf);

    rewind(tmpf);

    char buf[8];
    while (fgets(buf, sizeof buf, tmpf) != NULL)
        printf("\n%s\n", buf);

    if (feof(tmpf))
        puts("End of file reached");
}
```



文本文件操作 – 字符串读或写

读数据:

char *fgets(char *str, int count, FILE *stream);

- 从文件流读取最多 count - 1 个字符存储于 str 字符数组。若遇文件尾或换行符则终止分析。
- **返回值** 成功时 str, 失败时 NULL 。
- 注意:**gets(str)** 与 **fgets(str, count, stdin);**的语义区别。

写数据

int fputs(const char *str, FILE *stream);

- 熟悉的 puts 等价于 **fputs(str, stdout);** 。
- **返回值** 成功时, 返回非负值。失败时, 返回 EOF 并设置错误指示。



文本文件操作 – 字符读或写

读数据:

int fgetc(FILE *stream);

- 熟悉的 **getchar** (~~非~~etc) 等价于 **fgetc(stdin);**。
- 从给定的输入流读取下一个字符。
- **返回值** 成功时获得 **unsigned char** 并转换为 **int** 的字符，失败时为 EOF 。

写数据

int fputc(int ch, FILE *stream);

- 熟悉的 **putchar** 等价于 **fputc(stdout);** 。
- **返回值** 成功时返回写入的字符。失败时返回 EOF 。



文本文件操作 – 案例 – 字符读写与错误处理

```
/*text error*/
int main(void) {
    FILE* fp = fopen("test.txt", "r"); // 请自己创建测试文本文件，写一些汉字试试！
    if(!fp) {
        perror("File opening failed");
        return EXIT_FAILURE;
    }

    int c; // 注意: int, 非 char, 要求处理EOF
    while ((c = fgetc(fp)) != EOF) { // 标准 C I/O 读取文件循环
        putchar(c);
    }

    if (ferror(fp))
        puts("I/O error when reading");
    else if (feof(fp))
        puts("End of file reached successfully");

    fclose(fp);
}
```



二进制文件操作

二进制文件通常指程序把内存中数据**直接按特定顺序结构**输出到磁盘。二进制文件基本操作只有 `fread` 和 `fwrite` 两个操作。

1. 二进制文本是 `byte` 流，默认按块从磁盘读入或输出，即块缓冲：

- 例如：为了确保数据写入磁盘，有时需显式调用 `fflush` 同步流与磁盘文件的内容；

2. 二进制磁盘文件通常数据比较多，不只需要顺序读写，更多时候需要读写其中部分内容，即需要更新模式的支持：

- 例如：`dest = fopen("1.bin", "ab+")`；在1.bin文件后添加数据，“+”表示可以修改更正前面的数据；



二进制文件操作 – 顺序写数据

```
/*text to bin*/
int stBinWriter(int cnt, Student *st) {
    FILE *pf = fopen("student.bin","wb");
    if (!pf) {
        perror("Open File:");
        exit(10);
    }

    fwrite(&cnt, sizeof(int), 1, pf);
    int result = fwrite(st, sizeof(*st), cnt, pf);
    fclose(pf);
    return result;
}
```

请查看 strudent.txt 和 student.bin 的大小，简述存成二进制文件的优点与劣势。



文件的定位

默认情况下，对文件的读写是**按顺序从头到尾**进行的。

我们可以认为有一个**位置指针**

，它指出了在文件中读写的位置。这个指针每读/写完一个数据项后，就自动移动到下一个位置上。

有时，我们可能需要控制这个指针的位置，以一种我们自己设计的顺序来读/写文件。

这就是文件的定位。



文件位置指针的复位

rewind(fp_ptr)

将程序的**文件位置指针(file position pointer)**重新定位

在fp_ptr所指向的**文件的开头**(即字节编号为0)。

文件位置指针并不是一个指针，而是一个整数值，这个整数值表示文件中下一个读/写操作将发生在哪个字节位置上。所以，文件位置指针有时也被成为**文件偏移量(file offset)**。文件位置指针是与每一个文件相联系的FILE结构体中的一个成员。



文件与流

fseek

函数可以将文件位置指针定位在某个特定的位置上，从而实现对文件特定位置的读/写。

对于某个文件里面的数值：1 2 3 4 5 6 7 8 9

10，假设只希望读取1、3、5、7、9。

在读取了1后，位置指针自动移动到2的位置上，这时就应该调用fseek改变位置指针使其移动到3的位置上，便可以略过2



运行程序屏幕将显示:

1 3 5 7 9 0 0 0 0 0

```
int a[10]={1,2,3,4,5,6,7,8,9,10};  
int b[10]={0};  
int i;  
FILE* fp1, *fp2;
```

```
fp1=fopen( "c:\\1.dat", "wb" );  
fwrite(a, sizeof(int), 10, fp1);  
fclose( fp1 );
```

```
fp2=fopen( "c:\\1.dat", "rb" );  
for( i = 0; i < 5; i++ )  
{  
    fread( &b[i], sizeof(int), 1, fp2 );  
    fseek( fp2, sizeof(int), SEEK_CUR );  
}  
fclose( fp2 );
```

```
for ( i = 0; i < 10; i++) printf( "%d ", b[i] );
```



文件与流

fseek的函数原型是：

```
int fseek (FILE *stream, long int offset, int whence)
```

其中，offset是stream所指向文件的、从whence位置开始、文件位置指针将要搜索经过的字节数。表示文件位置指针的搜索起始位置的实参whence有三个可选值：
SEEK_SET表示从文件的**开头位置**开始搜索，**SEEK_CUR**表示从文件的**当前位置**开始搜索，**SEEK_END**表示从文件的**末尾**开始搜索



注意

利用fopen打开文件，若不是追加形式（没有a），则位置指针将处于文件头。

若带有“a”，则位置指针将处于文件尾。每次读写操作后，位置指针将自动移动到下一个位置。

进行定位的时候，就必须小心计算好相应于起始点要移动多少个字节，不要犯下多移少移的错误。



以字符形式读写文件（练习）

例子1：在屏幕上显示 D:\\demo.txt文件的内容

```
#include<stdio.h>
int main(){
    FILE *fp;
    char ch;
    // 如果文件不存在，给出提示并退出
    if( (fp=fopen("D:\\demo.txt","rt")) == NULL )
    {
        puts("Fail to open file!");
        exit(0);
    }
    // 每次读取一个字节，直到读取完毕
    while( (ch=fgetc(fp)) != EOF ){
        putchar(ch);
    }
    putchar("\n"); // 输出换行符
    fclose(fp);
    return 0;
}
```

例子2：从键盘向文件输入一行字符，写入文件

```
#include<stdio.h>
int main(){
    FILE *fp;
    char ch;
    // 判断文件是否成功打开
    if( (fp=fopen("D:\\demo.txt","wt+")) == NULL ){
        puts("Fail to open file!");
        exit(0);
    }
    printf("Input a string:\n");
    // 每次从键盘读取一个字符并写入文件
    while ( (ch=getchar()) != '\n' ){
        fputc(ch,fp);
    }
    fclose(fp);
    return 0;
}
```



以字符串形式读写文件（练习）

例子1：逐行打印文件 D:\\demo.txt 文件的内容

```
#include <stdio.h>
#include <stdlib.h>
#define N 100
int main(){
    FILE *fp;
    char str[N+1];
    if( (fp=fopen("d:\\demo.txt","rt")) == NULL ){
        puts("Fail to open file!");
        exit(0);
    }
    while(fgets(str, N, fp) != NULL){
        printf("%s", str);
    }
    fclose(fp);
    return 0;
}
```

例子2：输入一个字符串并追加到文件 D:\\demo.txt 的末尾

```
#include<stdio.h>
int main(){
    FILE *fp;
    char str[102] = {0}, strTemp[100];
    if( (fp=fopen("D:\\demo.txt", "at+")) == NULL ){
        puts("Fail to open file!");
        exit(0);
    }
    printf("Input a string:");
    gets(strTemp);
    strcat(str, "\\n");
    strcat(str, strTemp);
    fputs(str, fp);
    fclose(fp);
    return 0;
}
```



以数据块的形式读写文件（练习）

例子1：从键盘输入一个数组，将数组写入文件再读取出来。

```
#include<stdio.h>
#define N 5
int main(){
    // 从键盘输入的数据放入a，从文件读取的数据放入b
    int a[N], b[N];
    int i, size = sizeof(int);
    FILE *fp;
    if( (fp=fopen("D:\\demo.txt", "rb+")) == NULL ){
        // 以二进制方式打开
        puts("Fail to open file!");
        exit(0);
    }
    // 从键盘输入数据 并保存到数组a
    for(i=0; i<N; i++){
        scanf("%d", &a[i]);
    }
    // 接→_→右边
```

```
    // 接左边
    // 将数组a的内容写入到文件
    fwrite(a, size, N, fp);
    // 将文件中的位置指针重新定位到文件开头
    rewind(fp);
    // 从文件读取内容并保存到数组b
    fread(b, size, N, fp);
    // 在屏幕上显示数组b的内容
    for(i=0; i<N; i++){
        printf("%d ", b[i]);
    }
    printf("\n");
    fclose(fp);
    return 0;
}
```

运行结果：

```
23 409 500 100 222✓
23 409 500 100 222
```



随机读写文件（练习）

例子1：从键盘输入三组学生信息，保存到文件中，然后读取第二个学生的信息。

```
#include<stdio.h>
#define N 3
struct stu{
    char name[10];    // 姓名
    int num;          // 学号
    int age;          // 年龄
    float score;      // 成绩
}boys[N], boy, *pboys;
int main(){
    FILE *fp;
    int i;
    pboys = boys;
    if( (fp=fopen("d:\\demo.txt", "wb+")) == NULL ){
        printf("Cannot open file, press any key to exit!\n");
        getch();
        exit(1);
    }
    // 接→_→右边
```

```
    // 接左边
    printf("Input data:\n");
    for(i=0; i<N; i++,pboys++){
        scanf("%s %d %d %f", pboys->name, &pboys->num, &pboys->age, &pboys->score);
    }
    fwrite(boys, sizeof(struct stu), N, fp);    // 写入三条学生信息
    fseek(fp, sizeof(struct stu), SEEK_SET);    // 移动位置指针
    fread(&boy, sizeof(struct stu), 1, fp);    // 读取一条学生信息
    printf("%s %d %d %f\n", boy.name, boy.num, boy.age, boy.score);
    fclose(fp);
    return 0;
}
```

运行结果：

Input data:

Tom 2 15 90.5 ✓

Hua 1 14 99 ✓

Zhao 10 16 95.5 ✓

Hua 1 14 99.000000